Classification

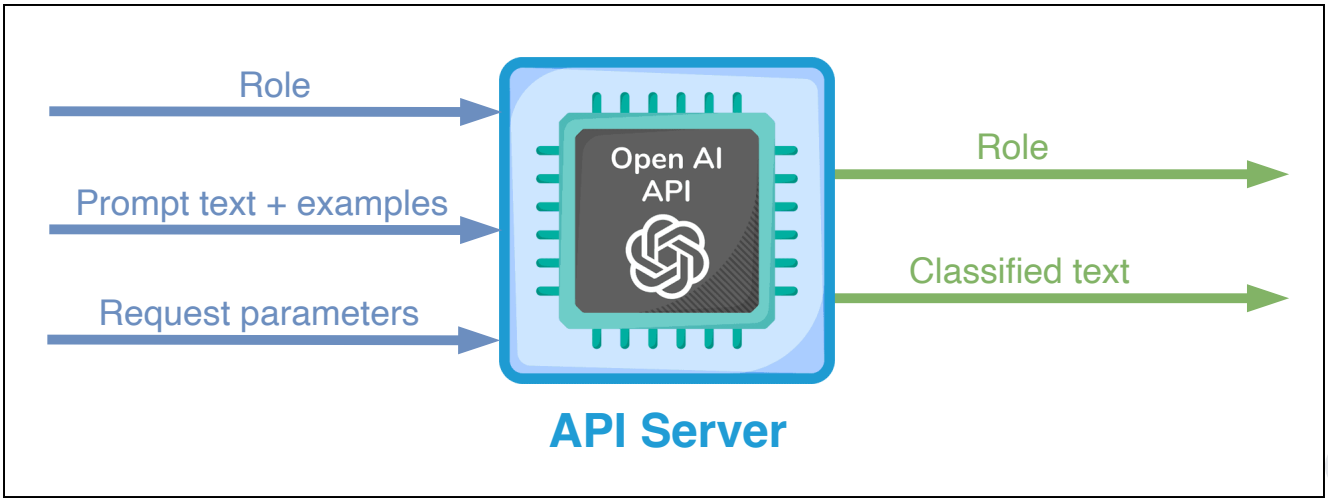
Explore how to classify text using OpenAI's completions endpoint with Python. Learn to design prompts for sentiment analysis and label multiple tweets effectively. Understand the impact of temperature and top_p on model responses and apply classification to practical NLP tasks like sentiment detection.

Overview

Classification is the process of categorizing the data into groups. In this lesson, we'll focus on the text classification problem. There are many applications of text classification, including sentiment analysis, natural language inference (NLI), and grammatical verification. We'll use the chat completions endpoint of OpenAI to classify the text data, which can be used to label any text without fine-tuning the model. To classify the text, we provide the task description and some optional examples with labels.

Classification

We input the prompt text and request parameters into the OpenAI API's model to classify text.



Completions: classify text

Single tweet classifier

The input parameters are the same as we saw in the previous lesson. Let's see how to provide a prompt to the API for classification:

```
Classify the Tweet's label as positive, neutral, or negative.

Tweet: I loved the new Spiderman movie!
Sentiment:
```

The first sentence of the prompt is the description of what type of task to perform. We can also provide labels here to get the specific outputs, like we have provided the labels `positive`, `neutral` and `negative` in the prompt. Then, in the prompt, we use simple terms for the input and the required output, like `Tweet` and `Sentiment,` respectively. If there is a new thing that the API is unfamiliar with, we have to provide examples in such cases.

Ask

Examples

Let's try the prompt above in the code widget below:

Please provide values for the following:

SECRET_KEY Not Specified...

Edit Import Values from JSON

main.py Python

```python
# Classifying text using completions endpoint

from openai import OpenAI
client = OpenAI(api_key="{{SECRET_KEY}}")

response = client.chat.completions.create(
  model="gpt-3.5-turbo",
  response_format={ "type": "json_object" },
  messages=[
    {"role": "system", "content": ("You are a helpful assistant designed
                + "Classify the Tweet's label as positive, neutral, or ne
    {"role": "user", "content": ("Tweet: I loved the new Spiderman movie!
                + "Sentiment:")}
```

Run

Classifying text using the completions endpoint

We can also provide examples to the API and then ask the system to generate the response for us. Let's understand this using the code below:

Please provide values for the following:

SECRET_KEY Not Specified...

Edit Import Values from JSON Ask

main.py Python

```
# Classifying text using completions endpoint

from openai import OpenAI
client = OpenAI(api_key="{{SECRET_KEY}}")

response = client.chat.completions.create(
  model="gpt-3.5-turbo",
  response_format={ "type": "json_object" },
  messages=[
    {"role": "system", "content": ("You are a helpful assistant designed
                + "Classify the Tweet's label as positive, neutral, or ne
                + "Provide the label for the tweet where it hasn't been p
```

Run

Classifying text using the completions endpoint

Between **lines 13** and **16**, we've adjusted the user prompt to include examples reflecting the desired labels for the first three tweets. Consequently, we allow the API to generate the label for the last tweet.

We must use the `temperature` and `top_p` values carefully. For example, look at the following text with no labels:

```
Classify the Tweet.

Tweet: I loved the new Spiderman movie!
```

For the prompt example above, try out different `temperature` and `top_p` values. Changing either `temperature` or `top_p` is recommended, but not both simultaneously. If you change the value of `temperature`, then use the default value of `top_p`. We recommend trying the following values of `temperature`: `1`, `0.9`, `0.75`, `0.5`, `0.25`, `0`.

Ask

Please provide values for the following:

SECRET_KEY Not Specified...

Edit Import Values from JSON

main.py Python

```python
# Using the temperature parameter

from openai import OpenAI
client = OpenAI(api_key="{{SECRET_KEY}}")

response = client.chat.completions.create(
  model="gpt-3.5-turbo",
  response_format={ "type": "json_object" },
  messages=[
    {"role": "system", "content": ("You are a helpful assistant designed
                + "Classify the Tweet")},
    {"role": "user", "content": ("Tweet: I loved the new Spiderman movie!
                + "Sentiment:")}
```

Run

Using the temperature parameter

The model will return a more generic response for greater `temperature` values and a more definite response for lower `temperature` values.

Now, try different `engines` for the same prompt and examine the results. We recommend the engines mentioned in this lesson.

Multiple tweets classifier

Now that we understand how to design a classifier using the completions endpoint, let's use it to label multiple tweets using a single API call.

Ask

Example

The `prompt_text` given in the code widget below is an example of handling multiple examples in one API call.

Please provide values for the following:

SECRET_KEY Not Specified...

Edit Import Values from JSON

main.py Python

```python
# Advanced tweet classifier

from openai import OpenAI
client = OpenAI(api_key="{{SECRET_KEY}}")

response = client.chat.completions.create(
  model="gpt-3.5-turbo",
  response_format={ "type": "json_object" },
  messages=[
    {"role": "system", "content": "Classify the sentiment in the tweets.
    {"role": "user", "content": ("Classify the sentiment in these tweets:
                    + "1. \"I enjoy this program\"\n"
                    + "2. \"I don't like him anymore\"\n"
```

Run

Advanced tweet classifier

In the code above, we use `max_tokens` as `150` for the response because the number of response tokens can be greater than the default value, `16`. We can also try different values of the temperature parameter and observe the difference. In **line 19**, we have added a final message, `"Tweet sentiment ratings:"` to the user messages array. This message helps clarify the purpose or expectation of the analysis.

Classify items into categories

Ask

The OpenAI API is great at classifying items into discrete categories.

Example

Let's see an example of how to provide the prompt to categorize multiple items:

Please provide values for the following:

SECRET_KEY Not Specified...

Edit Import Values from JSON

main.py Python

```
from openai import OpenAI
client = OpenAI(api_key="{{SECRET_KEY}}")

response = client.chat.completions.create(
  model="gpt-3.5-turbo",
  response_format={ "type": "json_object" },
  messages=[
    {"role": "system", "content": "You are a helpful assistant designed t
    {"role": "user", "content": "The following is a list of companies. Fi
  ],
  temperature=0,
  max_tokens=100,
  top_p=1.0
```

Run

Classifying items into categories

We set the `temperature` to `0` because we need a well-defined answer and `top_p` with the value `1.0` to include all the tokens in the sampling.

Practical use case

Ask

A practical use case for OpenAI's text generation capabilities is to classify user sentiments expressed in messages as positive, negative, or neutral. Sentiment

analysis can provide valuable insights into customer satisfaction levels, enabling businesses to tailor responses and improve overall customer experience.

Mark As Completed

Ask